Universidade Estadual de Londrina

Ciência da Computação – Estrutura de Dados II (1COP010)
Prof. Ma. Vanessa Matias Leite

Prova 1

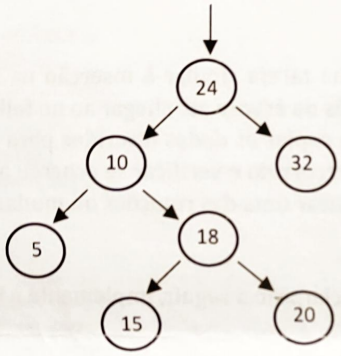
Aluno: _____

Data: 14 de março de 2023 **Nota**: _____ / 10,0

Instruções:

- *As questões seguintes deverão ser resolvidas na linguagem C.*
- *Não serão consideradas as folhas de rascunho.*
- *Algoritmos sem indentação serão desconsiderados.*

1) *(1,5)* Analise a árvore AVL a seguir.



Implemente a função de rotação que realize o balanceamento para o tipo de problema apresentado na árvore anterior. Além disso, realize o ajuste o fator de balanceamento. Para a sua implementação, considere a declaração a seguir:

```
#include <stdio.h>
#include <stdlib.h>
#define true 1
#define false 0

typedef int bool;
typedef int TIPOCHAVE;


typedef struct aux{
    TIPOCHAVE chave;
    struct aux *esq;
    struct aux *dir;
    int fb; // fator de balanceamento
} NO, *PONT;
```

Após a implementação da(s) função(ões), desenhe a árvore após o balanceamento.

2) *(3,5)* As árvores Rubro-Negras (RN) possuem características específicas que estão relacionadas a seguir:

1. Todo nó é vermelho ou preto.

2. A raiz é preta.

3. Toda folha (NULL) é preta.

4. Se um nó é vermelho, então os seus filhos são pretos.

5. Para cada nó, todos os caminhos simples do nó até folhas descendentes contêm o mesmo número de nós pretos.

Inserir um novo nó em uma árvore rubro-negra é uma tarefa similar à inserção na árvore AVL. Primeiramente, temos que percorrer um conjunto de nós da árvore até chegar ao nó folha que irá se tornar o pai do novo nó, alocar memória para este nó e copiar os dados inseridos para dentro dele. Uma vez inserido o nó, devemos voltar pelo caminho percorrido e verificar se ocorreu a violação de alguma das propriedades da árvore e, se necessário, aplicar uma das rotações ou mudança de cores para restabelecer o balanceamento da árvore.

Com base nas características das árvores RN e com a declaração a seguir, implemente a inserção em uma árvore RN.

```
#include <stdio.h>
#include <stdlib.h>

#define RED 1
#define BLACK 0

struct NO{
    int info;
    struct NO *esq;
    struct NO *dir;
    int cor;

};
```

* Implemente todas as funções que podem fazer parte da função de inserção.

3) *(1,0)* Mostre a árvore B de ordem 4 que resulta de carregar os seguintes conjuntos de chaves em ordem:

- C G J X
- C G J X N S U O A E B H I
- C G J X N S U O A E B H I F
- C G J X N S U O A E B H I F K L Q R T U W Z

4) *(2,5)* Para todas as questões desta prova que envolvam implementação de código utilizando hash, considere a declaração e as funções abaixo:

```
#include <stdio.h>
#include <stdlib.h>

//Tamanho da tabela
#define M 25

//Tipo pessoa
typedef struct{
    int codigo;
    char nome[50];
} Pessoa;


// tipo nó usado na lista encadeada
typedef struct no{
    Pessoa pessoa;
    struct no *prox;

} No;

//tipo lista com um ponteiro para o primeiro nó
typedef struct{
    No *inicio;
    int tam;
} Lista;


//Tabela (vetor de ponteiro para listas)
Lista *tabela[M]
```

Desenvolva as seguintes funções para a tabela hash:

- Criar a tabela hash;
- Calcular a posição da chave (função a sua escolha);
- Busca;
- Inserção;
- Imprimir tabela hash;

5) *(1,5)* Para todas as questões desta prova que envolvam implementação de código utilizando heaps, considere a declaração e as funções abaixo:

```
typedef struct heap {
  int *h;      /* Vetor de inteiros a ser alocado
                  no momento da inicialização do heap */
  int n;       /* Número de elementos no heap */
  int max_n;   /* Número máximo de elementos */
} Heap;

int pai(int i) { return (i-1)/2; }
int esq(int i) { return 2*i + 1; }
int dir(int i) { return 2*i + 2; }

int existe_pai(int i) { return i > 0; }
int existe_esq(int i, int n) { return esq(i) < n; }
int existe_dir(int i, int n) { return dir(i) < n; }
```

a) Escreva uma função que retorna o valor mínimo em um heap de máximo. Apenas as posições do vetor que podem armazenar o elemento mínimo devem ser visitadas (evite comparações desnecessárias). A função deve retornar -1 caso o heap esteja vazio.

b) Escreva uma função que retorna 1 caso o heap passado como argumento respeite as restrições de heap de máximo (para todo elemento que tenha pelo menos um filho, seu valor associado deve ser maior do que o do(s) seu(s) filho(s)) e 0 caso contrário.